Practical Assignment: - 2

Roll No: - 01

Name: - Barad Meet M.

Class: - MCA

Division: - Y1

Subject: - Advanced Database Management System

Create Database and Collection

Task 1: Create a database named CollegeDB.
Answer :- **use CollegeDB**

Task 2: Create a collection named students
Answer :- db.createCollection("students");

Task 3: Display the available collections.
Answer :- show dbs

Task 4: Insert one student record using insertOne().
Answer :- db.students.insertOne({rollno: 101,name: "Rahul",age: 22,course: "MCA",semester: 1,city: "Rajkot",marks: 85})

Task 5: Insert the following five student records using insertMany().
Answer :-

```
db.students.insertMany([{rollno:102,name:"Priya",age:23,course:"MCA",semester:1,city:"Ahmedabad",mark:90},
                {rollno:103,name:"Amit",age:22,course:"MCA",semester:3,city:"Rajkot",mark:75},
                {rollno:103,name:"Neha",age:24,course:"MCA",semester:3,city:"Surat",mark:88},
                {rollno:105,name:"Karan",age:21,course:"MCA",semester:1,city:"Jamnagar",mark:65},
                {rollno:106,name:"Riya",age:23,course:"MCA",semester:3,city:"Rajkot",mark:92}
               ])
```

Read Operations

Task 6: Display all student records.
Answer :- db.students.find()

Task 7: Display the details of the student whose name is "Rahul"..
Answer :- db.students.findOne({name:"Rahul"})

Task 8: Display all students belonging to Rajkot.
Answer :- db.students.find({city:"Rajkot"})

Task 9: Display students having marks greater than 80.
Answer :- db.students.find({mark:{$gt:80}})

Task 10: Display students having marks less than 70.
Answer :- db.students.find({mark:{$lt:70}})

Task 11: Display students whose age is greater than or equal to 23.
Answer :- db.students.find({age:{$gte:23}})

Task 12: Display students studying in semester 3.
Answer :- db.students.find({semester:3})

Task 13: Display only the name and marks of all students.
Answer :- db.students.find({},{name:1,mark:1,_id:0})

Task 14: Display students whose marks are between 70 and 90.
Answer :- db.students.find({mark: {$gte: 70, $lte: 90}})

Task 15: Display students belonging to Rajkot OR Surat.
Answer :- db.students.find({$or:[{city:"Rajkot"},{city:"Surat"}]})

Task 16: Display students whose semester is 3 AND marks are greater than 80.
Answer :- db.students.find({ mark: { $gt: 80 }, semester:3});

Task 17: Display students in descending order of marks
Answer :- db.students.find().sort({mark: -1})

Task 18: Display the top three students based on marks.
Answer :- db.students.find().sort({mark:-1}).limit(3)

Task 19: Count the total number of students..
Answer :- db.students.countDocuments({})

Task 20: Display students whose names start with "R".

Answer :- db.students.find({name: /^R/})
db.students.find({name: {$regex: "^R"}})

Update Operation

Task 1: Update Rahul's marks from 85 to 95.

Answer :- db.students.updateOne({name:"Rahul"},{$set:{mark:95}})

Task 2: Change Priya's city to Mumbai.

Answer :- db.students.updateOne({name:"Priya"},{$set:{city:"Mumbia"}})

Task 3: Increase marks of all students by 5.

Answer :- db.students.updateMany({},{$inc:{mark:5}})

Task 4: Add a new field status: "Active" to all students.

Answer :- db.students.updateMany({},{$set:{status:"Active"}})

Task 5: Change the semester of all semester 1 students to semester 2.

Answer :- db.students.updateMany({},{$set:{status:"Active"}})

Task 6: Rename the field marks to totalMarks for all students.

Answer :- db.students.updateMany({},{$rename:{mark:"totalmark"}})

Task 7: Remove the age field from Rahul's document.

Answer :- db.students.updateMany({},{$rename:{mark:"totalmark"}})

Task 8: Replace Karan's entire document with new details.

Answer :-
bb.students.replaceOne({name:"Karan"},{rollno:105,age:22,course:"MCA",semester:2,city:"Surat",totalmark:80})

Task 9: Update Riya's city to Vadodara and semester to 4.

Answer :- db.students.updateOne({name:"Riya"},{$set:{city:"Vadodara",semester:4}})

Task 10: Increase the age of all students by 1.

Answer :- db.students.updateMany({},{$inc:{age:1}})

Delete Operation

Task 1: Delete the student record whose name is "Amit".

Answer :- db.students.deleteOne({name:"Amit"})

Task 2: Delete one student belonging to Surat.

Answer :- db.students.deleteOne({city:"Surat"})

Task 3: Delete all students whose semester is 4.

Answer :- db.students.deleteMany({semester:4})

Task 4: Delete all students having totalMarks less than 75.

Answer :- db.students.deleteMany({totalmark:{$lt:75}})

Task 5: Delete all students belonging to Rajkot.

Answer :- db.students.deleteMany({city:"Rajkot"})

Task 6: Delete all records from the students collection without deleting the collection.

Answer :- db.students.deleteMany({})

Task 7: Delete the students collection completely.

Answer :- db.students.drop()

ASSIGNMENT EXERCISES

Level – 1 Basic

```
use LibraryDB

db.createCollection("books")

db.books.insertMany([
  {
    book_id: 101,
    title: "Mastering Java",
    author: "James Gosling",
    price: 650,
    category: "Programming",
    quantity: 10
  },
  {
    book_id: 102,
    title: "MongoDB Basics",
    author: "John Smith",
    price: 450,
    category: "Database",
    quantity: 8
  },
  {
    book_id: 103,
```

```
    title: "Python Programming",
    author: "Guido van Rossum",
    price: 750,
    category: "Programming",
    quantity: 4
},
{
    book_id: 104,
    title: "Computer Networks",
    author: "Andrew Tanenbaum",
    price: 600,
    category: "Networking",
    quantity: 6
},
{
    book_id: 105,
    title: "MySQL Guide",
    author: "Paul DuBois",
    price: 350,
    category: "Database",
    quantity: 3
},
{
    book_id: 106,
    title: "Modern C++",
    author: "Bjarne Stroustrup",
    price: 850,
    category: "Programming",
    quantity: 7
},
{
    book_id: 107,
    title: "Network Security",
    author: "Charlie Kaufman",
    price: 550,
    category: "Networking",
    quantity: 0
},
{
    book_id: 108,
    title: "Machine Learning",
    author: "Tom Mitchell",
    price: 700,
    category: "Programming",
    quantity: 5
},
{
    book_id: 109,
```

```
    title: "SQL Fundamentals",
    author: "John J. Patrick",
    price: 180,
    category: "Database",
    quantity: 12
  },
  {
    book_id: 110,
    title: "Web Development",
    author: "Jon Duckett",
    price: 280,
    category: "Programming",
    quantity: 2
  }
]);
```

```
db.books.find()
```

```
db.books.find({price:{$gt:500}})
```

```
db.books.find({category:"Programming"})
```

```
db.books.find({},{title:1,author:1,_id:0});
```

```
db.books.countDocuments()
```

Level – 2 Intermediate

```
db.books.find({price:{$gt:300,$lt:700}})
```

```
db.books.find({quantity:{$lt:5}})
```

```
db.books.find().sort({ price: 1 });
```

```
db.books.find().sort({ price: -1 }).limit(3);
```

```
db.books.updateMany({ category: "Programming" },{ $inc: { price: 100 } });

db.books.updateOne({ book_id: 101 },{ $set: { quantity: 15 } });

db.books.updateMany( {},{ $set: { availability: "Yes" } });

db.books.updateMany({},{ $rename: { "price": "bookPrice" } });

db.books.updateOne({ book_id: 110 },{ $unset: { category: "" } });

db.books.replaceOne(
   { book_id: 101 },
   {
      book_id: 101,
      title: "Advanced Java Programming",
      author: "Herbert Schildt",
      bookPrice: 900,
      quantity: 15,
      availability: "Yes"
   }
);
```

Level – 3 Advanced

```
db.books.find({category: "Programming",bookPrice: { $gt: 500 }});

db.books.find({category: { $in: ["Database", "Networking"] }});

db.books.find({title: { $regex: "^M" }});

db.books.updateMany({},{ $inc: { quantity: 10 } });

db.books.updateOne(
   { book_id: 111 },
   {
```

```
      $set: {
         title: "Learning JavaScript",
         author: "John Resig",
         bookPrice: 500,
         category: "Programming",
         quantity: 10,
         availability: "Yes"
      }
   },
   { upsert: true }
);

db.books.deleteOne({quantity: 0});

db.books.deleteMany({bookPrice: { $lt: 20 }});
```